Cahier des Charges Fonctionnel

Amélioration Volant Fanatec

BERTAULT Nolhan

Année universitaire : 2026-2027

Sommaire

Sommaire 2
Mise en contexte 4
I. Le simulateur complet 4
II. Volant Fanatec BMW 5
III. Base du volant Fanatec CSL DD QR2 (5NM) 6
IV. Fonctions Principales 6
V. Expression du besoin 7
VI. Contraintes initiales 7
VII. Parties prenantes 7
VIII. Éléments additionnels 7
I.I. LEDs RPM 7
I.II. Boutons/Potentiomètres 8
I.III. USB-C 9
Électronique 10
I. Composants 10
I.I. Microcontrôleur 10
I.II. Convertisseur USB-UART 10
I.III. ASM1117 10
I.IV. Diode 10
I.V. Level shifter 10
I.VI. Quartz 10
I.VII. Résistances pour les boutons 11
II. Contraintes 11
III. Broches 12
IV. Câblage Microcontrôleur 13
V. Éléments nécessaires 14
VI. Mapping des pins ATmega328P-AU 15
VI.I. Connexion base -> PCB 15
VI.II. Connexions PCB –> BMW 16
VI.III. Connexions PCB -> éléments supp. 16
VII. Carte électronique 17
VII.I. Schématique 17
VII.II. PCB 18
Firmware 19

I. Structure du code 19
II. Contenu des trames 20
Mécanique 21
I. Contraintes 21
II. Face Avant du Modèle 3D 22
II.I. Plan général 22
II.II. Espaces boutons d'origine 23
II.III. Espaces boutons, potentiomètres et joystick 23
II.IV. Emplacement des LEDs 24
II.V. Fixation des cartes 24
Carte d'origine 24
Carte PCB 24
II.VI. Résumé des contraintes de la face avant 24
III. Face Arrière du Modèle 3D 25
III.I. Module arrière d'origine 25
III.II. Face arrière 25
Résultat Final 26
Annexe 27
Protocole MISO/MOSI 27
Sources 28

Mise en contexte

I. Le simulateur complet

Le simracing occupe aujourd'hui une place de plus en plus importante dans le marché du divertissement numérique. Cette discipline, qui consiste à reproduire fidèlement les sensations de la conduite automobile à l'aide d'équipements spécialisés, attire un public toujours plus large. Toutefois, malgré une volonté croissante de rendre cette pratique accessible au plus grand nombre, certaines ressources demeurent relativement coûteuses. En effet, pour profiter d'une expérience immersive et réaliste, il est souvent nécessaire d'investir dans du matériel de bonne qualité, ce qui peut représenter un frein pour de nombreux utilisateurs.

C'est à partir de ce constat que nous avons eu l'idée de **concevoir une amélioration** destinée à l'un des volants les plus abordables de la marque Fanatec. L'objectif est de proposer une solution permettant d'optimiser les performances de cet équipement tout en **conservant un coût raisonnable**, afin de rendre le simracing plus accessible sans sacrifier la qualité de l'expérience.

Une simulation complète et immersive repose sur plusieurs éléments essentiels. On retrouve tout d'abord le volant, qui constitue l'interface principale entre l'utilisateur et le simulateur. À cela s'ajoutent les pédales (accélérateur, frein et embrayage) qui permettent de reproduire fidèlement les actions de conduite. Un levier de vitesse peut également être intégré pour renforcer le réalisme. Enfin, l'élément central de l'ensemble est la base. Celle-ci joue un rôle fondamental puisqu'elle récupère les différentes informations envoyées par les composants du simulateur, puis les convertit en données exploitables par l'ordinateur. Connectée en USB, la base assure la communication entre tous les périphériques, permettant ainsi d'initialiser et de synchroniser le volant, les pédales et le levier de vitesse pour garantir une expérience fluide et cohérente.

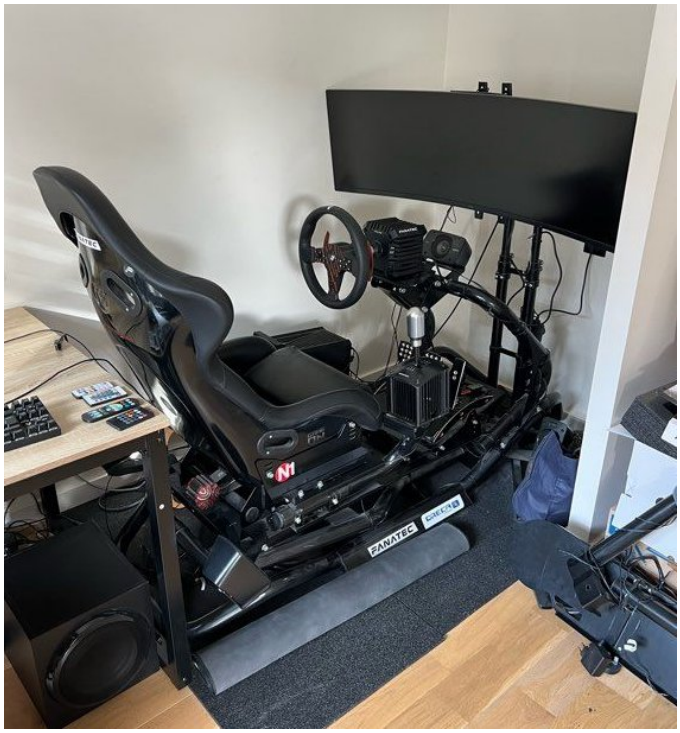


Image 1 : Simulateur complet de SimRacing

II. Volant Fanatec BMW

Ce volant se présente comme un modèle rond relativement classique, conçu pour offrir une prise en main simple et efficace. Il est équipé de plusieurs commandes intégrées, dont un joystick à quatre positions, une LED d'indication, un afficheur LCD, ainsi que 13 boutons configurables. À cela s'ajoutent deux palettes magnétiques situées à l'arrière, utilisées pour le passage des rapports, offrant une sensation plus nette et réactive lors de l'utilisation.

Avec un diamètre de 30 cm, ce volant propose des dimensions proches de celles d'un volant automobile standard, ce qui contribue à une certaine immersion. Sa conception repose sur un mélange de matériaux comprenant principalement du caoutchouc pour le confort de prise en main, de l'aluminium pour la structure, et du plastique pour certains éléments externes.



Image 2 : Volant Fanatec BMW

L'objectif de ce projet est de faire évoluer ce volant relativement simple vers un modèle plus abouti, se rapprochant davantage des volants utilisés en compétition automobile. Pour cela, nous souhaitons nous inspirer des volants présents dans les voitures de catégorie GT3, reconnues pour leur ergonomie et leur technicité. L'idée est d'y intégrer des fonctionnalités supplémentaires ainsi que des améliorations ergonomiques, afin de proposer une expérience de simulation plus immersive, plus réaliste et mieux adaptée aux exigences des passionnés de simracing.

III. Base du volant Fanatec CSL DD QR2 (5NM)



Image 3 : Fanatec Wheel Base CSL DD QR2 (5NM)

La base de volant Fanatec CSL DD QR2 (5 Nm) constitue l'élément central d'un simulateur de conduite. Elle joue un rôle essentiel en assurant la liaison entre les différents composants du volant (boutons, palettes, capteurs) et l'ordinateur. Concrètement, elle récupère toutes les informations provenant du volant et les transmet au système via une connexion USB, permettant ainsi leur exploitation par les jeux ou logiciels de simulation.

En plus de cette fonction de communication, la base traite également les données envoyées par le jeu, notamment celles liées au retour de force. Grâce à son moteur Direct Drive, elle convertit ces informations en sensations physiques directement retransmises dans le volant. Cela permet au pilote de ressentir les effets de la route, les pertes d'adhérence ou encore les vibrations, offrant une immersion beaucoup plus réaliste.

La base assure aussi la gestion de l'alimentation et de la coordination des différents périphériques connectés, comme les pédales ou le levier de vitesse. Elle centralise ainsi toutes les interactions entre le matériel et le logiciel, garantissant un fonctionnement fluide et cohérent de l'ensemble du simulateur. Enfin, grâce à sa connexion USB et à son électronique embarquée, elle agit comme une interface fiable et performante entre l'utilisateur et l'environnement virtuel. Elle permet non seulement de transmettre rapidement les données, mais aussi d'assurer une grande précision dans leur traitement, ce qui est indispensable pour une expérience de simulation de qualité.

IV. Fonctions Principales

Type	Fonction	Description
FP1	Permettre de diriger le véhicule	Traduire les mouvements du volant en commandes dans le jeu
FP2	Transmettre des commandes supplémentaires	Permettre l'utilisation de boutons, palettes ou encodeurs
FP3	Assurer le confort d'utilisation	Offrir une bonne prise en main et une ergonomie adaptée
FP4	Communiquer avec l'ordinateur	Envoyer les données via USB ou autre protocole
FC1	Être économique	Utiliser des composants peu coûteux
FC2	Être solide	Résister aux efforts et à une utilisation répétée
FC3	Être léger	Faciliter la manipulation et réduire la fatigue
FC4	Être personnalisable	Permettre des modifications ou améliorations
FC5	Être simple à fabriquer	Utiliser des pièces imprimables en 3D et des composants accessibles

Tableau 1 : Fonctions Principales

V. Expression du besoin

Le besoin se résume en quatre points :

- **Communiquer nativement avec la base Fanatec CSL DD** via le protocole SPI propriétaire, sans driver additionnel côté PC.
- **Ajouter des entrées physiques supplémentaires** (4 boutons, 2 potentiomètres) directement intégrés sur le PCB.
- **Afficher les shift lights RPM en temps réel** sur un bandeau LED SK6812, piloté depuis les données reçues de la base. En plus de l'afficheur 7 segments déjà présent sur le volant d'origine.
- **Conserver la carte Fanatec BMW M CSL d'origine** (palettes + boutons existants) en la relayant via un second bus SPI logiciel.

VI. Contraintes initiales

- **Alimentation :** le +5V doit provenir exclusivement du pin 4 du connecteur QR Fanatec. Aucune alimentation externe.
- **Niveaux logiques :** la base Fanatec fonctionne en 3.3V sur le bus SPI. L'ATmega328P fonctionnant en 5V, une conversion de niveau est requise sur la ligne MISO.
- **Protocole fermé :** le protocole SPI Fanatec est propriétaire. L'implémentation repose sur le reverse engineering communautaire.
- **Format PCB :** le PCB doit tenir à l'intérieur de la carrosserie imprimée en 3D, tout en ayant un espace confortable pour que l'utilisateur ai accès aux boutons.
- **Soudage à la main :** Tous les composants doivent être facilement mis en place et soudable à la carte car celle-ci sera réalisée à la main.

VII. Parties prenantes

Partie prenante	Rôle	Attente
Utilisateur	Pilote sim racing	Contrôles supplémentaires, retour visuel RPM, fiabilité
Base Fanatec CSL DD	Maître SPI, alimentation	Réponse SPI conforme (trame 12 oct. + CRC8)
Carte BMW M CSL	Esclave SPI secondaire	Être interrogée proprement, fonctionner sans modification
PC / Driver Fanatec	Reconnaissance périphérique	Voir un volant Fanatec standard, aucun driver supplémentaire

Tableau 2 : Récapitulatif acteurs

VIII. Éléments additionnels

I.I. LEDs RPM

Pour afficher les tours par minutes, ou RPM en anglais, nous allons, comme sur la majorité des volant de course, les afficher avec un bandeau de LED. Chaque LED indique un certain niveau de tour par minute, défini par le simulateur auquel le volant est relié. Celle-ci pourront changer de couleur, clignoté, etc. mais cela ne nous concerne pas pour la conception de la carte électronique. Pour la relier à la carte nous allons nous servir d'une connexion JST à 3 broches



Image 4 : Illustration LEDs

I.II. Boutons/Potentiomètres

Les boutons et potentiomètres sont directement soudés à la carte PCB, cela permettra d'avoir des éléments plus solide, une communication plus rapide et de simplifier les connections sur la carte, réduire le nombre de connecteurs. Chacun de ces éléments seront ensuite couvert d'un cache afin de rendre leur utilisation plus simplifié et plus esthétique.



Image 5 : Boutons pressoirs



Image 6 : Potentionmètre

I.III. USB-C

Afin de réaliser la partie codage il nous faut pouvoir communiquer avec le microcontrôleur. Pour cela nous avons opté pour un simple port USB-C, qui sera sécurisé par une diode en vu des différentes tension présentent dans le système.

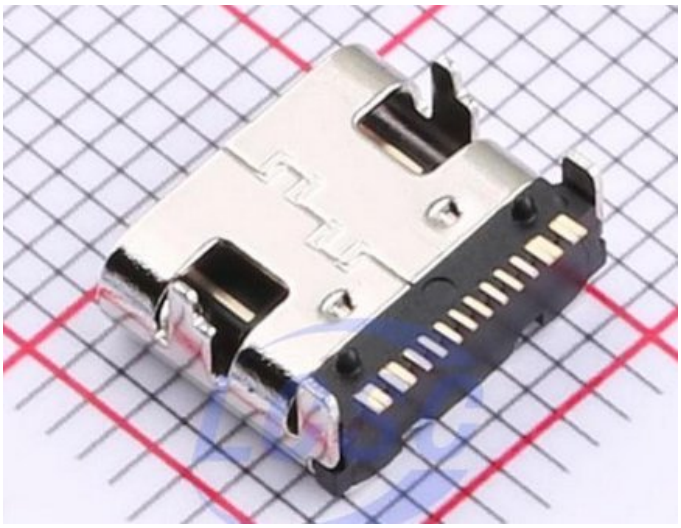


Image 7 : Connecteur USB-C

Électronique

I. Composants

I.I. Microcontrôleur

Le choix s'est porté sur l'ATmega328P-AU pour plusieurs raisons concrètes. D'abord c'est le microcontrôleur qui équipe les Arduino Nano et Uno — il est donc très bien documenté, la communauté sim racing l'a déjà validé sur ce type de projet (notamment lshachar et darknao), et l'IDE Arduino permet de le programmer sans apprendre un environnement de développement complexe. Ensuite il dispose nativement d'un contrôleur SPI hardware qui peut fonctionner en mode esclave, ce qui est indispensable pour répondre aux trames de la base Fanatec. Il a aussi suffisamment de pins pour tout câbler sans être à court, et son prix autour de 1.60€ sur LCSC est imbattable. Le format TQFP-32 a été choisi plutôt que le DIP-28 pour réduire la surface du PCB — un DIP-28 ferait presque doubler la taille de la carte.

I.II. Convertisseur USB-UART

Il existe plusieurs convertisseurs USB-UART comme le CP2102, le FT232, ou le CH340G. Le CH340C a été choisi pour deux raisons principales. La première est qu'il ne nécessite pas de quartz externe contrairement au CH340G — la version C intègre son oscillateur en interne, ce qui simplifie le schéma et réduit le nombre de composants. La seconde est son prix très bas (~0.35€ sur LCSC) et sa disponibilité. Les drivers Windows sont gratuits et fonctionnent bien. Le FT232 aurait été plus fiable mais coûte 10 fois plus cher pour le même usage.

I.III. ASM1117

Le pin 7 du connecteur Fanatec fournit effectivement du +3.3V. Cependant cette tension vient directement de l'alimentation interne de la base. Si on y branche directement la référence AREF de l'ATmega et les deux potentiomètres, on risque de perturber cette tension par notre consommation, ce qui peut déstabiliser le bus SPI de la base et provoquer des erreurs de communication. L'AMS1117-3.3 crée un +3.3V complètement isolé dérivé du +5V, uniquement pour notre PCB. C'est une bonne pratique qui garantit une référence ADC stable et propre pour nos potentiomètres.

I.IV. Diode

Pendant la phase de développement, nous allons brancher simultanément le PCB sur la base Fanatec (qui fournit +5V par le pin 4) ET sur votre PC via USB-C (qui fournit aussi +5V via le CH340C). Sans diode ces deux sources de +5V se retrouvent en court-circuit — l'une pousse dans l'autre ce qui peut endommager la base ou le port USB du PC. La diode laisse passer le courant dans un seul sens, protégeant les deux sources.

I.V. Level shifter

Le bus SPI de la base fonctionne en 3.3V. L'ATmega alimenté en 5V produit des signaux à 5V. Sur les lignes MOSI, SCK et CS qui vont de la base vers l'ATmega, ce n'est pas un problème — l'ATmega tolère des entrées en 3.3V sans difficulté, la tension est simplement interprétée comme un HIGH valide. En revanche sur la ligne MISO qui va de l'ATmega vers la base, le signal 5V arrive sur une entrée 3.3V de la base — c'est trop élevé et peut endommager l'électronique de la base sur le long terme. C'est pourquoi seule la ligne MISO nécessite une conversion de niveau de 5V vers 3.3V. On utilise soit un diviseur résistif (deux résistances) soit un level shifter dédié.

I.VI. Quartz

L'ATmega328P peut fonctionner avec son oscillateur interne à 8MHz, mais à 8MHz la fréquence SPI maximale est divisée par deux et la vitesse d'exécution du firmware est réduite. À 16MHz l'ATmega a 16 millions d'instructions par seconde disponibles — largement suffisant pour traiter les trames SPI, lire les boutons, calculer le CRC8 et mettre à jour les LEDs dans la fenêtre de 10ms. Le quartz SMD 3225 4 broches a été choisi pour sa taille compacte. Les condensateurs de charge de 22pF sont imposés par la datasheet de l'ATmega pour assurer la stabilité de l'oscillateur — leur valeur dépend du quartz utilisé, 22pF étant la valeur standard pour un quartz 16MHz.

I.VII. Résistances pour les boutons

Un bouton momentané est un simple interrupteur — quand il est ouvert, la pin de l'ATmega n'est reliée à rien du tout, elle flotte entre 0V et 5V de manière aléatoire selon les interférences électromagnétiques ambiantes. L'ATmega lirait des appuis fantômes en permanence. La résistance pull-up de 10kΩ relie la pin au +5V en permanence — quand le bouton est relâché la pin lit 5V (HIGH = pas d'appui), quand on appuie le bouton court-circuite la pin à GND et elle lit 0V (LOW = appui détecté). La valeur de 10kΩ est un compromis standard — assez faible pour maintenir la pin à un niveau stable, assez élevée pour ne pas consommer trop de courant quand le bouton est pressé (0.5mA à 5V/10kΩ).

II. Contraintes

Contraintes	Valeurs
Fréquence interrogation base	~100 Hz (10 ms/cycle)
Fréquence horloge SPI	~500 kHz
Durée trame complète	~0.72 ms
Latence max réponse firmware	< 10 ms
Latence réelle ATmega 16MHz	< 1 ms
Marge sécurité	> 9 ms
Mise à jour bandeau SK6812	~0.6 ms (20 LEDs)

Tableau 3 : Contraintes électroniques

III. Broches

Puisque nous souhaitons partir d'une base déjà existante, il a fallu étudier la manière dont les éléments du volant communiquent et sont traduits avant d'être transférés à la base. Nous avons donc recherché des projets similaires afin de mieux comprendre cette architecture, et nous avons trouvé le GitHub de darknao. Dans ce projet, il détaille précisément la méthode utilisée pour ajouter de nouveaux boutons à son volant, notamment en identifiant les broches du connecteur et leur rôle. Il explique également les composants électroniques nécessaires, comme les microcontrôleurs ou les circuits de gestion des entrées, ainsi que la façon dont les signaux sont interprétés puis envoyés à la base. Cette analyse nous a permis de mieux appréhender le fonctionnement global du système et de nous appuyer sur une démarche déjà validée pour envisager nos propres modifications.

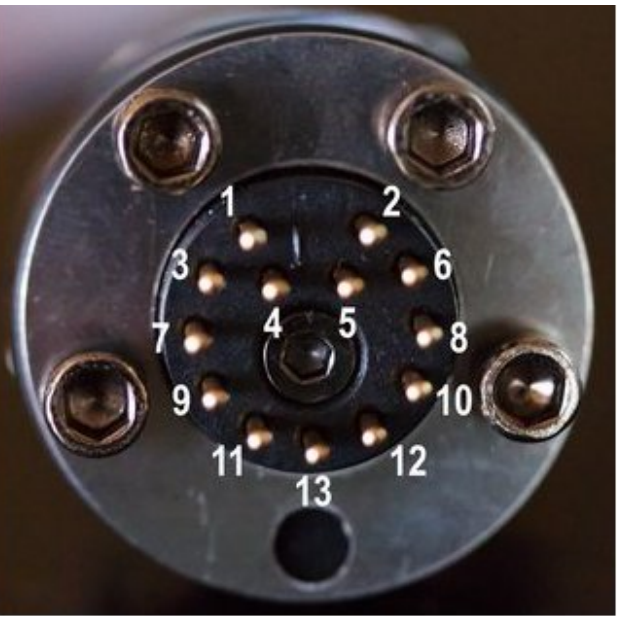


Image 8 : Connecteur vers Base

Pin	Signal	Niveau	Utilisé ?	Remarque
1	MISO	3.3V	Oui	Données esclave → base. Nécessite level shifter si MCU 5V
2	MOSI	3.3V	Oui	Données base → esclave
3	GND	—	Oui	Masse signal
4	+5V	5V	Oui	Alimentation — source unique du système DIY
5	GND	—	Oui	Masse puissance
6	SCK	3.3V	Oui	Horloge SPI (~500 kHz)
7	+3.3V	3.3V	Oui	Référence AREF pour les ADC / régulateur
8	CS / SS	3.3V	Oui	Chip Select + interruption INT0 (D2 + D10 reliés)
9	USB+	—	Non	USB interne (Universal Hub uniquement)
10	NC	—	Non	Non connecté
11	DP1	—	Non	Data Port 1 — USB secondaire
12	USB-	—	Non	USB interne (Universal Hub uniquement)
13	DP2	—	Non	Data Port 2 — USB secondaire

Tableau 4 : Connecteurs Broche vers Base

> ***Note :*** *Les pins 9 à 13 ne sont utilisés que par l'Universal Hub Fanatec pour un bus USB interne. Ils sont physiquement présents sur le connecteur (13 broches mécaniquement) mais électriquement non connectés sur ce PCB.*

On s'est donc aperçu que le système fonctionnait avec un protocole MISO (Master In Slave Out), alimenté en 5V. Cela signifie que la communication entre la base et le volant repose sur un échange de données série de type SPI, où la base joue le rôle de maître et le volant celui d'esclave. Les différentes broches identifiées, comme MOSI, SCK et CS, permettent respectivement l'envoi de données vers le volant, la synchronisation via une horloge, et la sélection du périphérique. L'alimentation en 5V assure le fonctionnement global du volant, tandis que certaines lignes supplémentaires, comme le 3,3V ou les ports de données, peuvent être utilisées pour alimenter les LEDs des RPM. Ce type de connexion permet une communication rapide et fiable, essentielle pour transmettre en temps réel les informations issues des boutons, palettes ou encodeurs présents sur le volant.

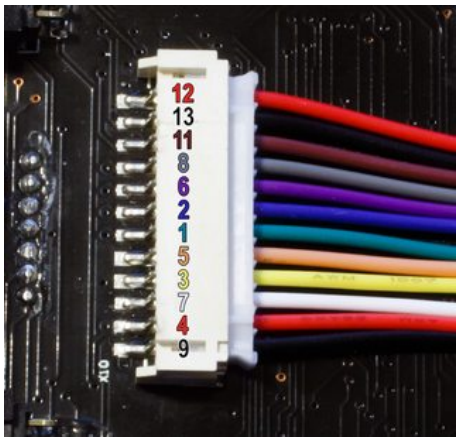


Image 9 : Port Broche Volant

Les broches sont reliés au à la carte PCB par une broche 13 ports. Ceux-ci sont représentés sur la figure ci-dessus, nous allons devoir nous en servir afin d'ajouter les éléments cités précédemment.

Dans notre cas seul les pins 1 à 8 nous seront utiles pour l'ajout des nouveaux éléments. Nous allons donc utiliser une connectique similaire afin de relier la carte PCB à la base et à la carte originale.

IV. Câblage Microcontrôleur

Afin de réduire au maximum la taille de la carte PCB, notre choix c'est porté sur une ATMega328P-AU, celle-ci compose habituellement les Arduino, c'est donc un composant peu coûteux, facile à codé et auquel nous avons déjà put faire face.

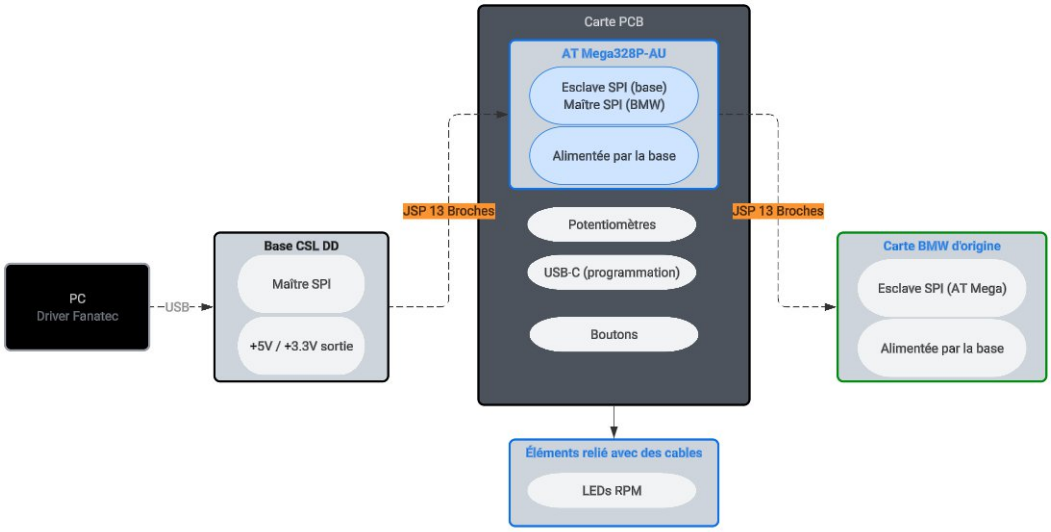


Figure 1 : Schéma de câblage

Avec ce schéma on peut s'apercevoir que l'ATMega est le corps principal de notre système, il permet la liaison entre les éléments importants, tout en laissant la liaison avec le PC à la base. Il va se servir des données reçues par la carte d'origine, y ajouter les informations de nos nouveaux éléments et va recréer une trame complète, et va les envoyer vers la base, qui de son côté la traduiras et l'enverras au PC.

V. Éléments nécessaires

Élément	Quantité	Lien
ATmega328p-AU	x1	https://tinyurl.com/4xssuzxa
CH340C	X1	https://tinyurl.com/ywyrej5m
AMS1117-3.3	X1	https://tinyurl.com/ycy4whrc
Connecteur USB-C femelle	X1	https://tinyurl.com/mt2z72te
Quartz 16MHz	X1	https://tinyurl.com/4udfjnw8
Diode 1N4148W	X1	Fournis
Résistances 0805	X5	Fournis
Condensateurs 0805	X11	Fournis
JST XH B12B-XH-A (lot)	X2	https://tinyurl.com/a5nsvzpa
JST XH B3B-XH-A (lot)	X1	https://shorturl.at/ShUqF
Bandeau LED SK6812	X1	https://shorturl.at/5hF6o
Boutons momentanés (Lots)	X4	https://shorturl.at/Aekwg
Capuchon de boutons (Lots)	X1	https://shorturl.at/arTn2
Potentiomètres linéaires 10kOhm	X2	https://shorturl.at/WL4XI
Capuchon de potentiomètres (Lots)	1	https://shorturl.at/7AqWY

Tableau 5 : Éléments à commander

VI. Mapping des pins ATmega328P-AU

Pin	Port Arduino	Signal	Direction	Fonction
1	D3 / PD3	MISO_BMW	Entrée	SPI soft MISO — lecture carte BMW
2	D4 / PD4	MOSI_BMW	Sortie	SPI soft MOSI — commande carte BMW
3	—	VCC	+5V	Alimentation MCU
4	—	GND	—	Masse
5	XTAL1	—	—	Quartz Y1 broche 1 (16 MHz)
6	XTAL2	—	—	Quartz Y1 broche 2 (16 MHz)
7	D5 / PD5	SCK_BMW	Sortie	SPI soft SCK — horloge vers carte BMW
8	D6 / PD6	CS_BMW	Sortie	SPI soft CS — sélection carte BMW
9	D7 / PD7	LED_DATA	Sortie	Signal données bandeau SK6812
10	D8 / PB0	BTN1	Entrée	Bouton 1 (pull-up 10kΩ intégrée)
11	D9 / PB1	BTN2	Entrée	Bouton 2 (pull-up 10kΩ intégrée)
12	D10 / PB2	CS_BASE	Entrée	SPI hard SS + relié à D2 (INT0)
13	D11 / PB3	MOSI_BASE	Entrée	SPI hard MOSI — données base
14	D12 / PB4	MISO_BASE	Sortie	SPI hard MISO — réponse vers base (via level shifter)
15	D13 / PB5	SCK_BASE	Entrée	SPI hard SCK — horloge base
16	—	AVCC	+5V	Alim. analogique + C découplage 100nF
18	—	AREF	+3.3V	Référence ADC (sortie AMS1117) + C découplage 100nF
19	—	GND	—	Masse
21	A0 / PC0	POT1	Entrée	Potentiomètre 1 — axe X (0–3.3V)
22	A1 / PC1	POT2	Entrée	Potentiomètre 2 — axe Y (0–3.3V)
23	A2 / PC2	BTN3	Entrée	Bouton 3 (pull-up 10kΩ intégrée)
24	A3 / PC3	BTN4	Entrée	Bouton 4 (pull-up 10kΩ intégrée)
25	A4 / PC4	—	—	Libre — I2C SDA (usage futur)
26	A5 / PC5	—	—	Libre — I2C SCL (usage futur)
27	RESET	nRESET	—	Pull-up R1 10kΩ + C9 100nF auto-reset
28	D0 / PD0	UART_RX	Entrée	RX depuis CH340C (TXD)
29	D1 / PD1	UART_TX	Sortie	TX vers CH340C (RXD)
30	D2 / PD2	CS_INT	Entrée	INT0 — interruption CS base (relié à D10)
31	—	GND	—	Masse
32	—	VCC	+5V	Alimentation MCU

Tableau 6 : Connectique AtMega328P

> ***Notes :*** *les broches D2 et D10 sont reliée afin de permettre le bon fonctionnement du système maître/esclave*

VI.I. Connexion base -> PCB

Pour la connectique en lien direct avec la base l'ATMega se contente de recevoir l'horloge, envoyer et recevoir les trames via le protocole MOSI/MISO et de définir l'esclave qui communique avec le maître, principe du protocole MISO/MOSI.

Port Arduino	Signal	Direction
D10 / PB2	CS_BASE	Entrée
D11 / PB3	MOSI_BASE	Entrée
D12 / PB4	MISO_BASE	Sortie
D13 / PB5	SCK_BASE	Entrée

Tableau 7 : Connectique ATMega vers Base

VI.II. Connexions PCB –> BMW

La connectique en lien avec la carte BMW d'origine reprend le même principe que celle avec la base, cependant la carte PCB devient alors maître tant dis que la carte BMW est l'esclave.

Port Arduino	Signal	Direction
D3 / PD3	MISO_BMW	Entrée
D4 / PD4	MOSI_BMW	Sortie
D5 / PD5	SCK_BMW	Sortie
D6 / PD6	CS_BMW	Sortie

Tableau 8 : Connectique ATMega vers carte BMW

VI.III. Connexions PCB -> éléments supp.

En ce qui concerne les éléments supplémentaire, la carte PCB récupère simplement les informations reçues des composants, sauf pour le bandeau LEDs où celle-ci ne fait qu'émettre afin de les faire s'allumer.

Port Arduino	Signal	Direction
D7 / PD7	LED_DATA	Sortie
D8 / PB0	BTN1	Entrée
D9 / PB1	BTN2	Entrée
A0 / PC0	POT1	Entrée
A1 / PC1	POT2	Entrée
A2 / PC2	BTN3	Entrée
A3 / PC3	BTN4	Entrée

Tableau 9 : Connectique ATMega vers éléments supplémentaires

VII. Carte électronique

L'objectif de notre carte PCB est de permettre une liaison plus fiable avec nos nouveaux éléments, les connexions par câble étant plus fragile, tout en permettant de limiter l'espace utiliser au sein de la coque imprimée. On placera alors les boutons et les potentiomètres directement sur cette carte, seul les LEDs, la base et la carte originale y sont connecté à l'aide de câbles (port JST).

VII.I. Schématique

Ce schéma sert simplement à définir les relation entre les composants et mettre en place de la structure de la carte électronique afin d'ensuite concevoir une carte PCB sur mesure.

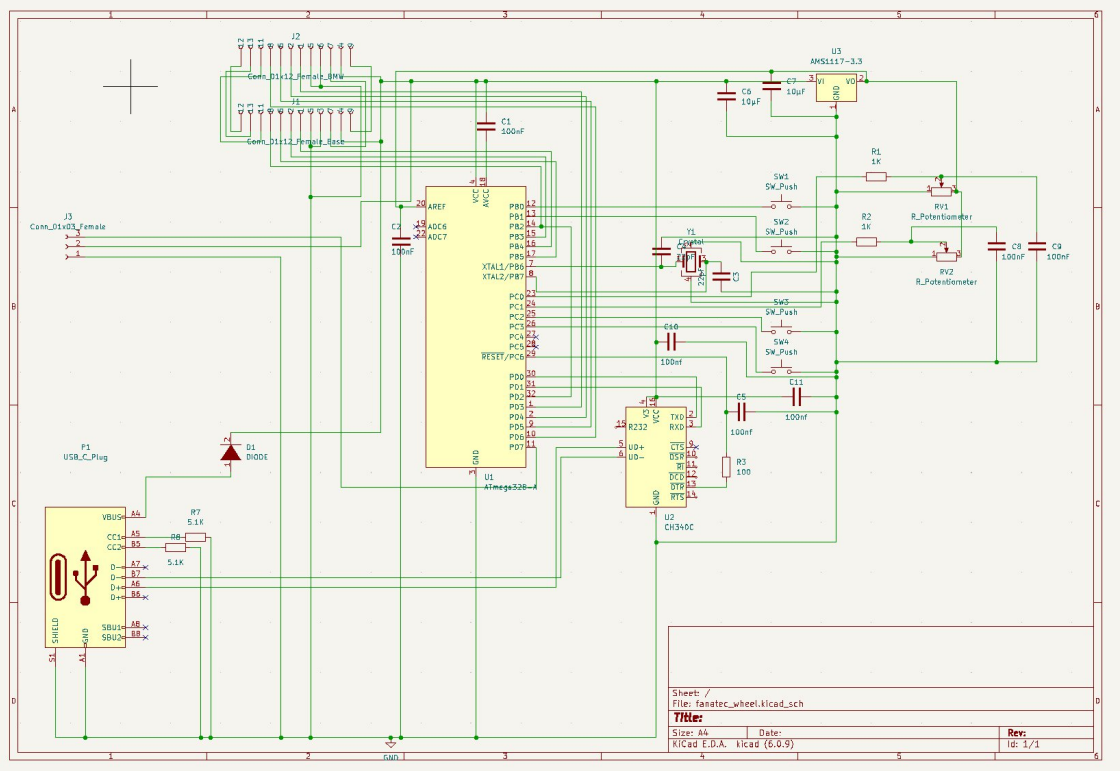


Figure 2 : Schéma de cablage éléments

VII.II. PCB

Après la réalisation du schéma précédent, nous pouvons désormais passer à la réalisation de la PCB. Les mesures affiché sont liés aux contraintes de taille du volant, mais aussi des annotations pour pouvoir adapter l'emplacement des éléments traversant le volant.

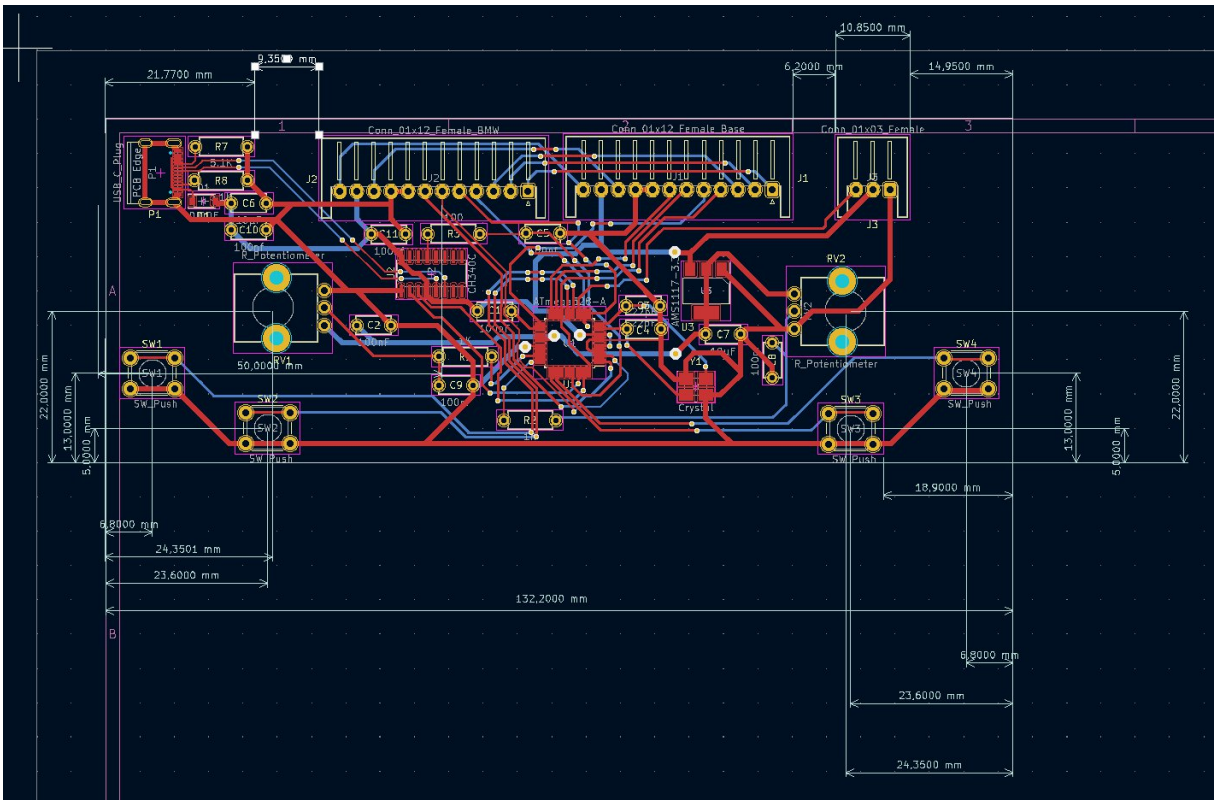


Figure 3 : Plan Carte PCB

Voila en image 3D la carte PCB qui sera imprimée. Les ports sont en haut pour faciliter la connexion des câbles, l'ATMega est au centre, la prise USB-C sur le coté gauche et les boutons et potentiomètres sur le bas de la carte.

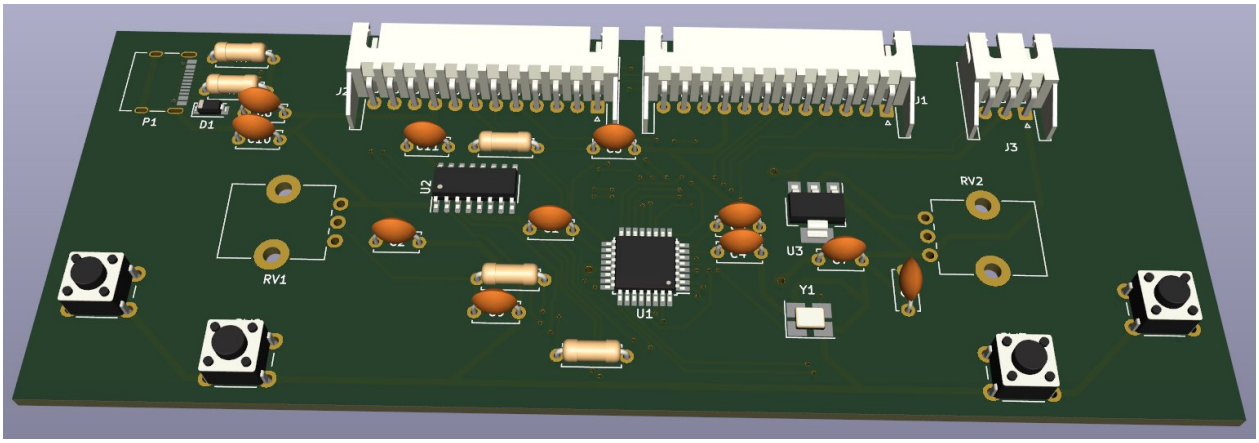


Image 10 : Plan 3D PCB

Firmware

Une grande partie du firmware est déjà codé sur l'ensemble des projet open sources dont nous nous sommes inspirés. Il ne nous reste plus qu'à ajouter les informations de nos nouveaux éléments. Ci-dessous un résumé des opérations à ajouter pour assurer le bon fonctionnement de nos nouveaux éléments.

I. Structure du code

Cette partie du code permet d'ajouter les composants, tel que nos boutons, nos potentiomètres et notre bandeau LED. Il suffira de rajouter cette partie au code déjà disponible dans les github.

```
void setup() {
  // Config SPI esclave
  SPCR |= _BV(SPE);   // Activer SPI
  SPCR &= ~_BV(MSTR); // Mode esclave
  SPCR |= _BV(SPIE);  // Interruptions SPI

  // CS sur D10 (hardware) ET D2 (interruption INT0)
  pinMode(10, INPUT); pinMode(2, INPUT);
  attachInterrupt(0, csChange, CHANGE);

  // Boutons : pull-up internes
  for (int i = 3; i <= 9; i++) {
    pinMode(i, INPUT_PULLUP);
  }
  // LEDs : sorties
  for (int i = 14; i <= 22; i++) {
    pinMode(i, OUTPUT);
  }
}

// ISR SPI : déclenché à chaque octet reçu
ISR(SPI_STC_vect) {
  byte received = SPDR;
  // Stocker l'octet entrant
  dataIn[rxIndex++] = received;
  // Préparer la réponse pour le prochain octet
  SPDR = dataOut[txIndex++];
}

void loop() {
  if (frameComplete) {
    // 1. Décoder les shift lights depuis dataIn[3..4]
    updateLEDs();
    // 2. Lire boutons et axes, encoder dans dataOut
    readButtons();
    readAxes();
    // 3. Calculer CRC8 et l'écrire en dataOut[11]
    dataOut[11] = crc8(dataOut, 11);
    frameComplete = false;
  }
}
```

Code 1 : Structure du code à ajouter

II. Contenu des trames

Octet(s)	Contenu	Valeur
0	En-tête Fixe	0xA5
1	ID Volant	0x04
2	Boutons bit 0-7	État des boutons 1–8
3	Boutons bit 8-15	État des boutons 9–16
4	Boutons bit 16-25	État des boutons 17–24
5	Axe X (joystick / pot.)	0–255
6	Axe Y	0–255
7	Encodeur	Valeur incrémentale
8-10	Réservé (PS buttons)	0x00
11	CRC8	Calculé sur octets 0–10

Tableau 10 : Valeurs Bits Trames

Mécanique

Dans la perspective d'améliorer le confort de la simulation, nous sommes parti dans l'idée de changer la forme de ce volant, le transformer en volant de GT/Hypercar. Pour cela nous avons conçu un modèle 3D permettant d'accueillir l'ensemble des éléments informatiques prévu précédemment en plus de la carte initiale. Il a aussi fallu penser à l'espace comprenant le connecteur à la base et les palettes.



Image 11 : Volant de GT de chez Fanatec

I. Contraintes

Afin d'avoir un volant correspondant aux normes ils nous faut nous imposer des contraintes. D'abord la taille, le volant doit pouvoir sembler confortable à l'utilisateur, en moyenne un volant de sport à un diamètre de 30cm, nous allons donc garder cette largeur en largeur maximum. Vu que notre volant n'a pas une forme circulaire, la hauteur sera réduite, il faut que celle-ci prenne en compte la taille des cartes PCB, les LEDs, de la place pour faire passer les connexions et enfin que l'arrière qui contient les palettes, et que l'on souhaite conserver puisse y être vissé. La longueur/épaisseur du volant est aussi un point assez important, pour éviter d'avoir un volant trop épais, on part sur la même que celui de base, soit 4cm. Cela laisse de l'espace pour que les connectiques ne soient pas compressé, et que les boutons puissent être relâchés et pressés sans avoir de risque d'appui non voulu.

Élément	Contraintes
Largeur du volant	~30cm
Hauteur du volant	~cm
Épaisseur du volant	4cm
Taille carte origine	
Taille carte PCB	5,02 x 13,22 cm

Tableau 11 : Contraintes du volant

II. Face Avant du Modèle 3D

II.I. Plan général

Nous avons donc repris la forme assez commune à la majorité des volants de ce style pour la face avant. On veut y intégrer des LEDs représentant les tr/min, de nouveaux boutons et des potentiomètres, en plus de ce qui est déjà présent sur le volant initial. Nous avons donc préparer une seconde carte PCB dans la partie Électronique, il nous faut donc prévoir un espace afin d'accueillir ces 2 éléments au sein du volant sans changer les dimensions.

Cette face est donc dans une forme horizontal, ressemblant presque à un rectangle. Il y a quatre espaces pour y insérer les doigts, et des rebords plus arrondi afin d'essayer d'épouser un minimum la forme des mains.

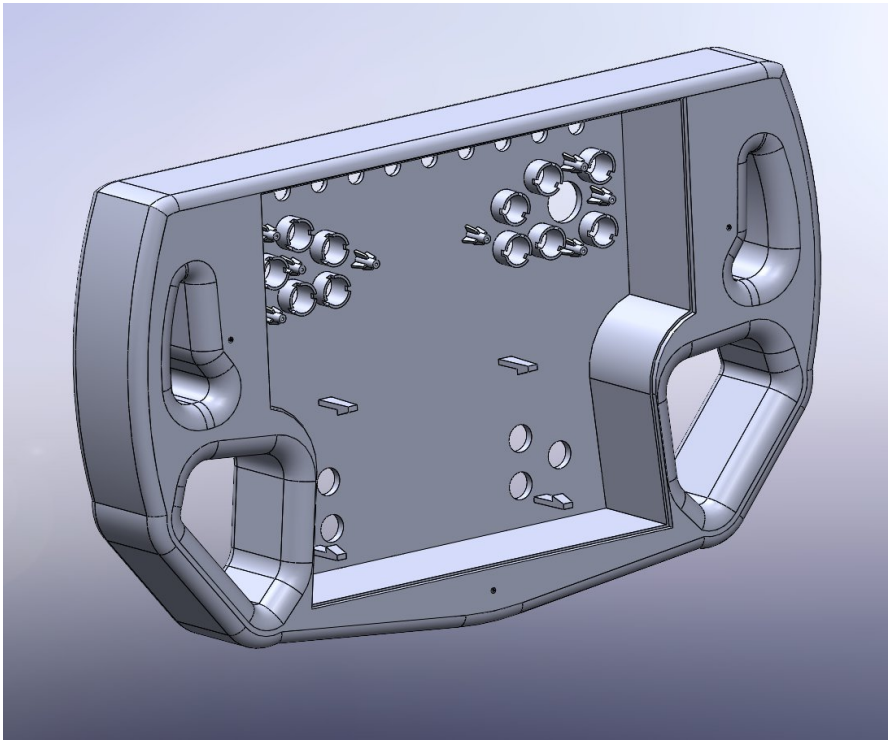


Image 12: Face avant du volant 3D (vue arrière)

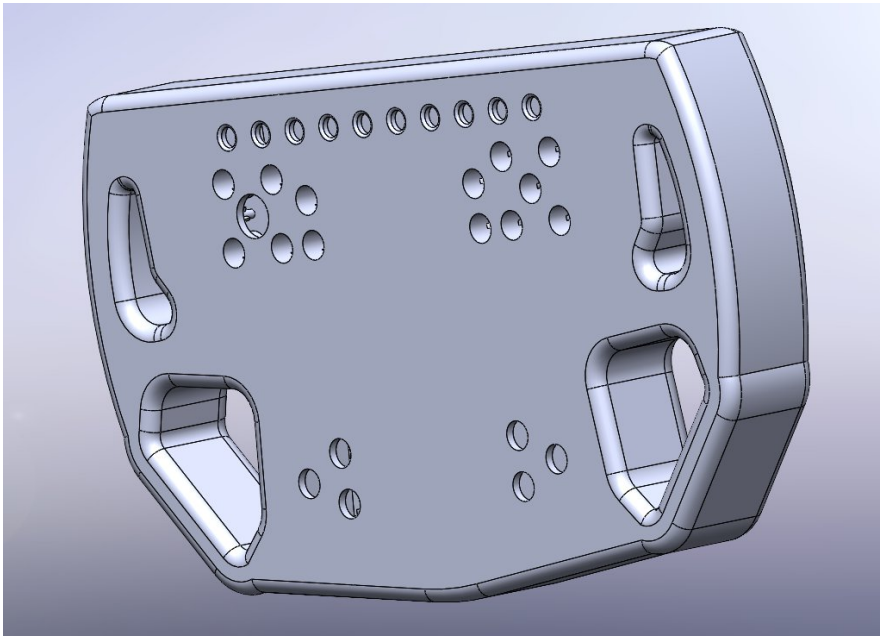


Image 13 : Face avant du volant 3D (vue avant)

II.II. Espaces boutons d'origine

Les boutons d'origine présent sur la carte avaient des espaces spécifiques pour qu'il soit parfaitement placés, avec des détrompeurs pour éviter que ceux-ci soient installés à l'envers ou qu'ils ne tournent sur eux même au fur et à mesure du temps.

II.III. Espaces boutons, potentiomètres et joystick

En ce qui concerne les autres éléments interactifs du volant il suffit de percer des trous du diamètre du bouton, cela réduit la charge de travail, bien qu'il y ai un risque que ceux-ci tournent sur eux même pour les boutons.

Voici un schéma reprenant les deux constat précédents.

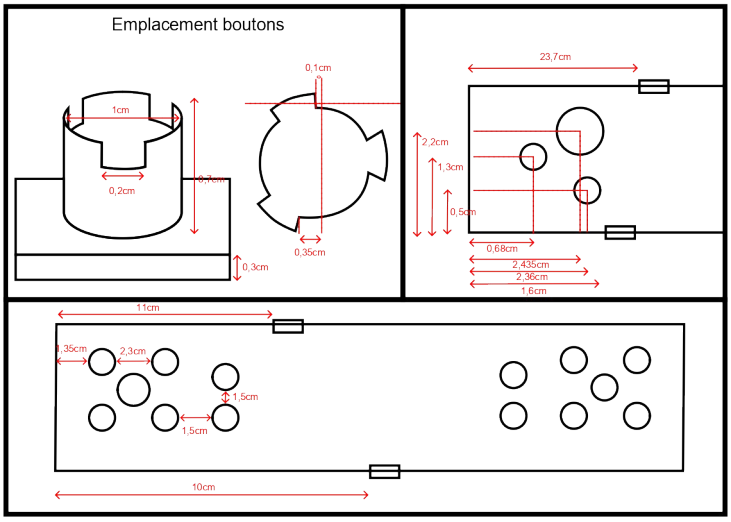


Figure 4 : Emplacements Boutons

II.IV. Emplacement des LEDs

Pour les LEDs, il a simplement fallu récupérer l'écart entre chaque LEDs et d'y installer des trous qui permettent de les voir.

II.V. Fixation des cartes

Carte d'origine

La carte d'origine est fixée avec des vis, dans ce contexte il nous faut prendre les mesures de leurs emplacements, et les intégrer dans notre modèle.

Carte PCB

Afin de simplifier le travail, la carte PCB sera fixé à l'aide de supports, il faudra juste qu'elle soit surélevé afin de ne pas serré les câbles, ou que les connectiques ne soient compressées et ne frottent contre le plastique.

II.VI. Résumé des contraintes de la face avant

Contraintes	
Espacement LEDs	12mm
Nombre de LEDs	10
Taille perçage boutons origine	1mm (diamètre)
Taille perçage joystick origine	1,5mm(diamètre)
Taille perçage bouton	1mm(diamètre)
Taille perçage potentiomètres	1mm(diamètre)
Espacement des cartes par rapport au modèle	1mm

Tableau 12 : Contraintes de la face avant

III. Face Arrière du Modèle 3D

Le volant initial à un bloc arrière qui contient l'emplacement pour les palettes et le système de connexion à la base, il nous faut donc adapter la forme arrière du nouveau volant afin de pouvoir y installer ce bloc. Puisque celui-ci à une forme triangulaire, il va falloir ajouter des plaques à l'arrière afin de protéger les composants et de les cacher.

III.I. Module arrière d'origine

Comme précisé précédemment le module arrière de base contient des éléments important et qui serais long à reproduire, on est donc parti du principe que l'on allez les conserver afin de réduire la charge de travail. Pour cela il nous faut adapter la forme de la face arrière et inclure des empreintes pour les vis dans la face avant.

III.II. Face arrière

La face arrière va se contenter de remplir les zones encore visible du système tout en permettant de faire circuler les connectiques. Celle-ci contient des feuillures afin de pouvoir réduire les mouvement parasites avec la face avant, mais celle-ci sera fixé à l'aide du module arrière d'origine qui sera vissé à la face avant

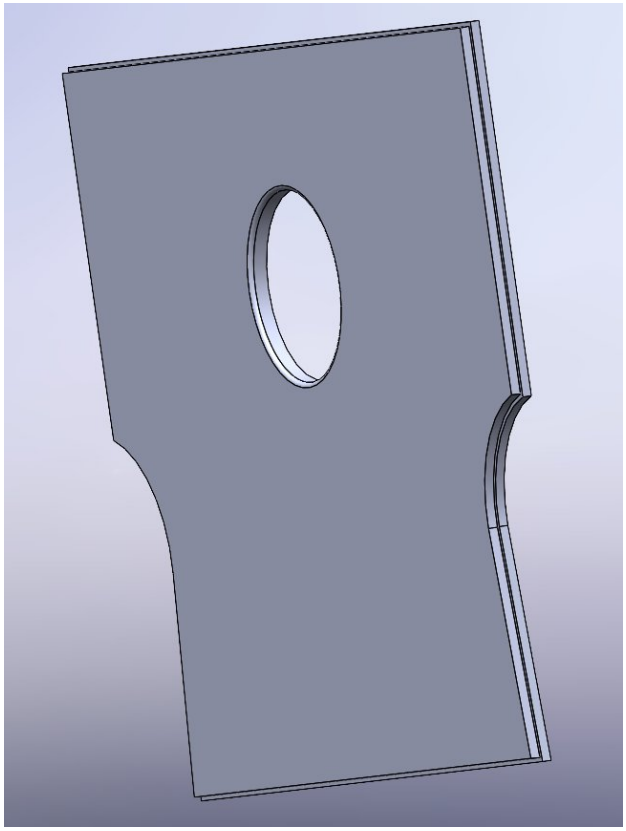


Image 14 : Face arrière

Résultat Final

Pour le moment le projet n'as pas put être réaliser à cause d'un manque de temps, d'une mauvaise organisation des cours. De mon côté je compte bien poursuivre le projet sur mon temps libre et en passant par des entreprises extérieurs. Cela rendra sûrement le projet un peu plus coûteux, mais il permettra une conception complète de celui-ci.

Annexe

Protocole MISO/MOSI

Le protocole MISO/MOSI, est composée de 4 signaux :

- MOSI : Le maître envoie les données vers l'esclave
- MISO : L'esclave envoie les données vers le maître
- SCK : horloge générée par le maître afin de synchroniser les échanges
- CS : Permet d'activer un esclave spécifique

Avec ces signaux le maître et ses esclaves peuvent communiquer de manière synchronisée, avec un maître qui peut choisir l'esclave avec lequel il est en communication. Le maître peut donc avoir plusieurs esclave, chacun ayant une ligne CS dédiée, qui est mis au niveau 0 lorsque le maître souhaite communiquer avec celui-ci. Ce protocole fonctionne en **full-duplex**, c'est à dire que les messages sont envoyés et reçus simultanément, d'où la raison d'une ligne MISO et une ligne MOSI. Ces messages sont envoyés à chaque tic (coup d'horloge), qui est définie par le maître.

Maître	Sens de la communication	Esclave
MOSI (envoi)	⟶	MOSI (reçu)
MISO (reçu)	⟵	MISO (envoi)
SCK (envoi)	horloge ⟶	SCK (reçu)
CS (envoi)	activé ⟶	CS (reçu)

Tableau 13 : Schéma explicatif MISO/MOSI

Note : Le protocole MOSI/MISO, peut aussi être nommé protocole SPI, les 2 noms sont utilisés dans ce document.

Sources

- **Arduino_Fanatec_Wheel** (lshachar) — Projet principal pour créer un volant DIY communiquant avec la base via SPI : https://github.com/lshachar/Arduino_Fanatec_Wheel
- **btClubSportWheel** (darknao) — Projet de reverse engineering du protocole SPI Fanatec. Contient le pinout complet et le code de capture de trames : https://github.com/darknao/btClubSportWheel
- **Tutoriel connecteur SPI mâle** — Comment fabriquer le connecteur côté volant (impression 3D + résine époxy) : https://github.com/lshachar/Arduino_Fanatec_Wheel/blob/master/Steering%20wheel%20connector%20-%20howto/Readme.md
- **Discussion protocole SPI (issue GitHub)** — Échanges communautaires très riches sur le protocole, le câblage et les problèmes d'EMI : https://github.com/darknao/btClubSportWheel/issues/12